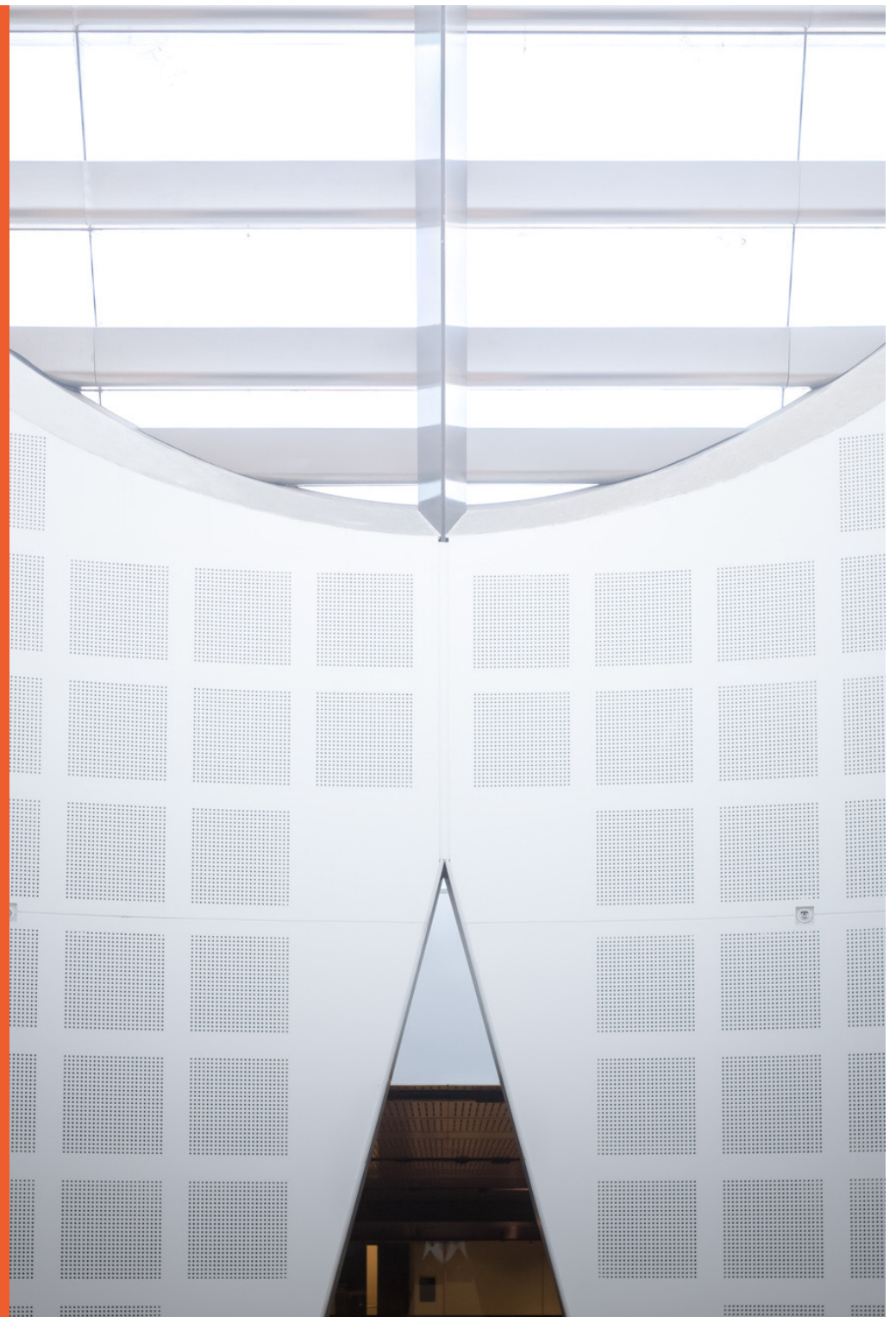


Material Review

Presented by
Dr. Huaming Chen

ELEC3609/9609
Internet Software Platforms
2025 S2



We acknowledge the tradition of
custodianship and law of the Country on which
the University of Sydney campuses stand.
We pay our respects to those who have cared
and continue to care for Country.



THE UNIVERSITY OF
SYDNEY

Overview

- ✓ **Web Fundamentals**
- ✓ **Scaling Web Architecture**
- ✓ **Web development process**
- ✓ **Web programming and testing**
- ✓ **Cloud services, CI/CD pipeline**
- ✓ **Web security**

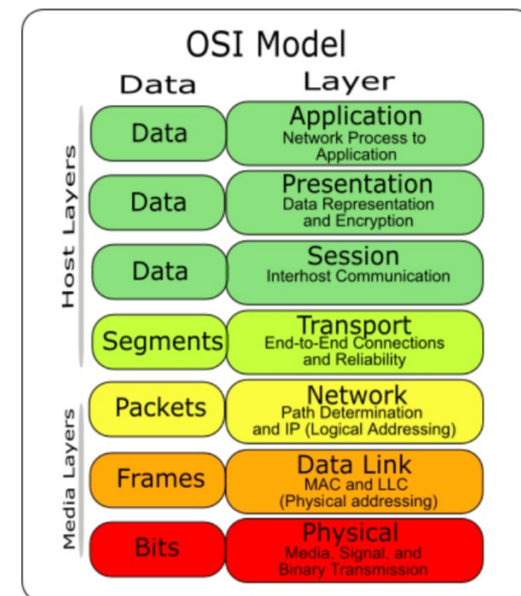
Web Fundamentals

Network Basic: OSI model

OSI is a way of conceptualising the network systems.

You need to be familiar with Layer 3, Layer 4 and Layer 7 throughout this course.

- Refreshing the understanding on the network basics
- Layer 1: Copper line, COAX, Fibre etc
- Layer 2: IEEE 802.11a/b/g/n
- Layer 4: TCP/UDP
- Layer 7: HTTPS



DHCP

- When you connect to a network, usually your computer is configured with the Dynamic Host Configuration Protocol (DHCP). This provides your computer with:
 - **IP address:** Identifies you on the local network
 - **Subnet Mask:** Defines what “local” means with regard to your IP address
 - **Default gateway:** A host that can route packets to non-local IP addresses
 - **Nameservers:** A host which can be used to resolve domain names – i.e., a DNS server
- First thing to check if you cannot reach any web sites

Other common web protocols

- DNS: translate URL to IP address
- SSH: allow web-based server administration and remote code execution
- BGP: border gateway protocol (interconnectivity and routing)

TCP and UDP

	TCP	UDP
Network Layer	4	4
Mechanisms	TCP guarantees in-order delivery of data for the application layer(s) above it. Hand shake. Famous cyber attack pattern: TCP flooding	UDP is a 'best effort' protocol, which sends 'datagrams' instead of TCP's 'segments'. With UDP, packets are not acknowledged or re-transmitted (unless performed explicitly at the application layer). No hand shake enforced.
Port designation	Needed e.g., SSH on TCP/22	Needed e.g., DNS on UDP/53

HTTP Methods

- These are the most commonly used HTTP methods:
 - GET: used to request data about a specified resource.
 - POST: used to create a resource.
- Many more exist:
 - PUT: create/update a resource.
 - PATCH: partial modification of a resource.
 - DELETE: deletes the specified resource.
 - OPTIONS: returns the supported HTTP methods for this resource.
 - HEAD: equivalent to a GET request without the response body.
- Consistency is key.

HTTP Cookies

- A HTTP cookie is a small piece of data, set by a website, which is stored in the user's browser.
- Browsers automatically include all the cookies set by a server with every HTTP request sent to that server.
- There are two HTTP headers which control cookies on the web:
- **Requests: Cookie**
Cookie: username=abc123; authToken=4f3e
- **Responses: Set-Cookie**
Set-Cookie: username=abc123
Set-Cookie: authToken=4f3e

HTTP Cookies II

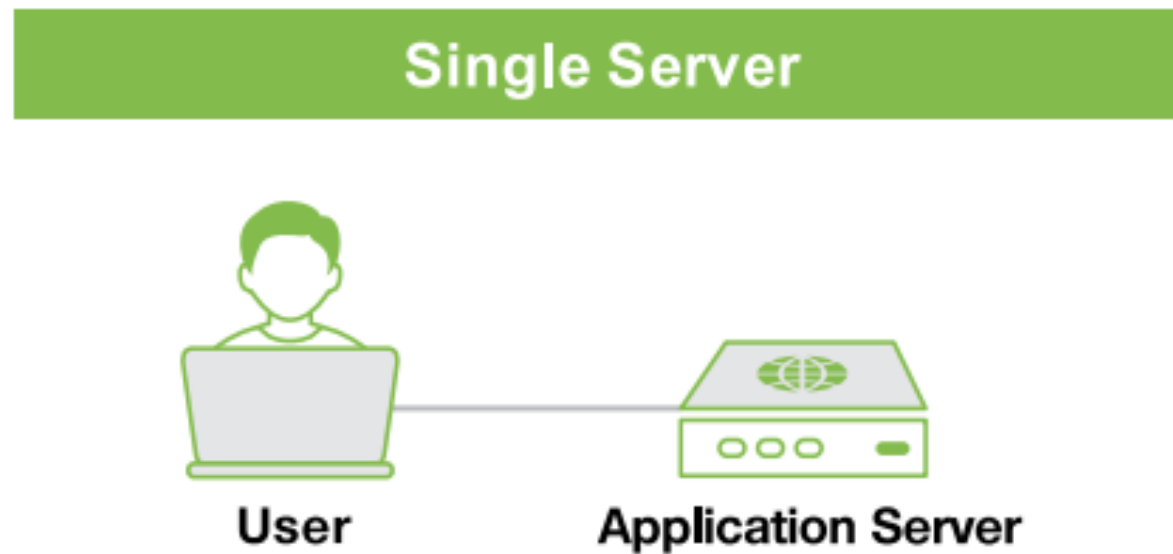
Cookies are often used for tracking users and authentication.

A typical login flow:

- 1. User visits website for first time.
 - no cookie
- 2. User submits login form to server with username + password.
 - still no cookie
- 3. Server generates authentication token (auth-tok).
 - Server responds with Set-Cookie: token=[auth-tok]
- 4. User visits website.
 - Client sends: Cookie: token=[auth-tok]
- 5. Server recognises (auth-tok) and knows request came from User.

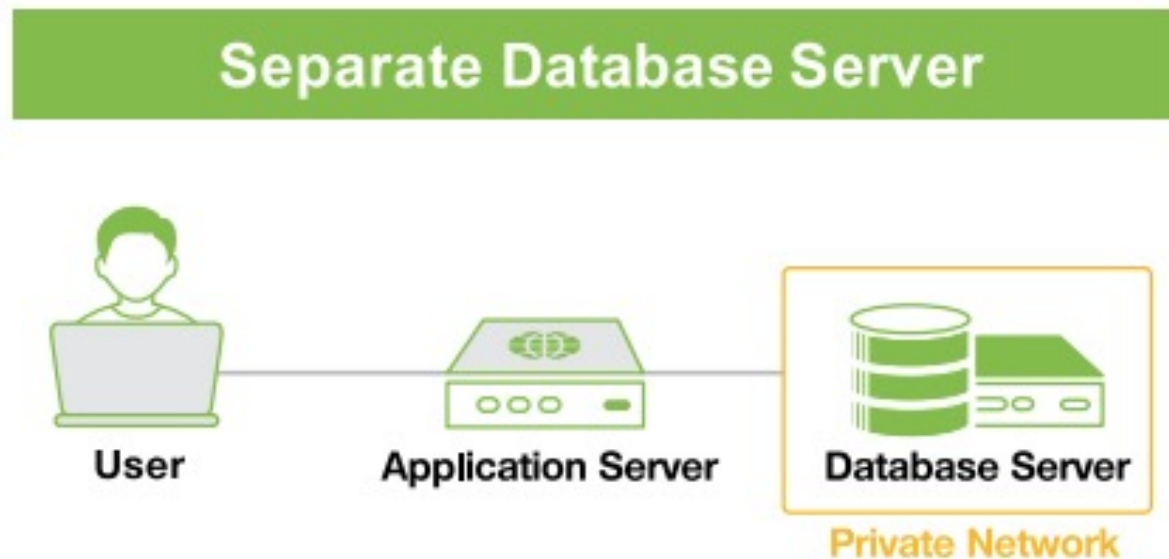
Scaling Web Architecture

Recap: Single Server Architecture



Separate DB Server

- Separate the database from the application server to eliminate resource contention. Increase security by removing the database from the DMZ, or public internet.



Separate DB Server

Pros

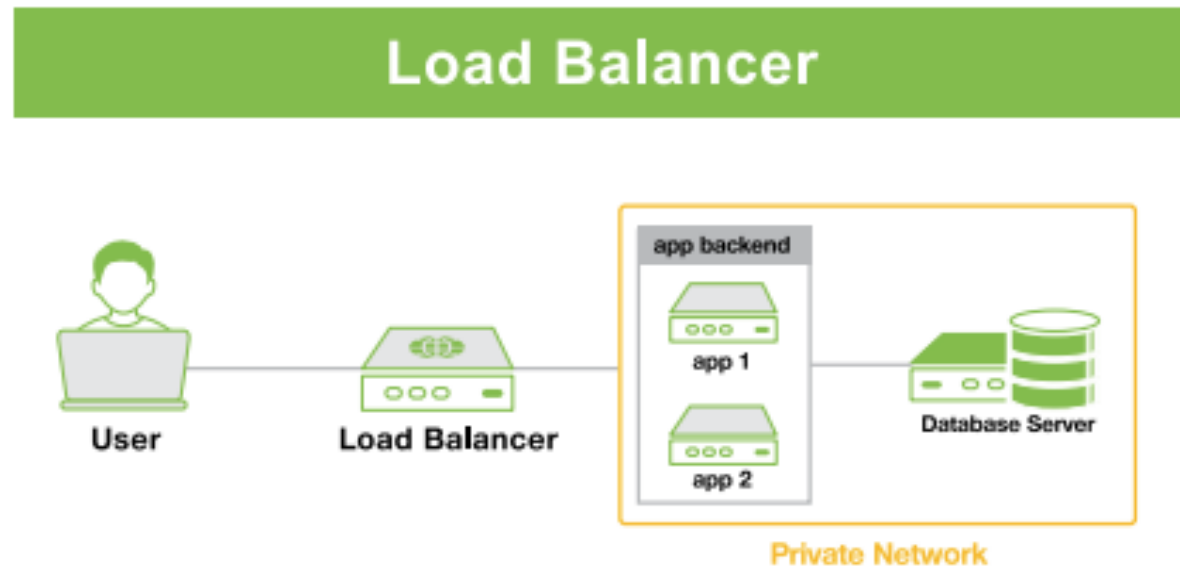
- ⦿ Application and database tiers do not contend for the same server resources (CPU, Memory, I/O, etc.)
- ⦿ You may vertically scale each tier separately
- ⦿ Can increase security by removing your database from the DMZ

Cons

- ⦿ More complex setup than single server
- ⦿ Performance issues can arise if the network connection between servers is poor

Load Balancer

- Load balancers (LB) can be added to a server environment to improve performance and reliability by distributing the workload across multiple servers. If one of the servers that is load balanced fails, the other servers will handle the incoming traffic until the failed server becomes healthy again.
- Typical implementations use HAProxy, Nginx, and Varnish.



Load Balancer

Pros

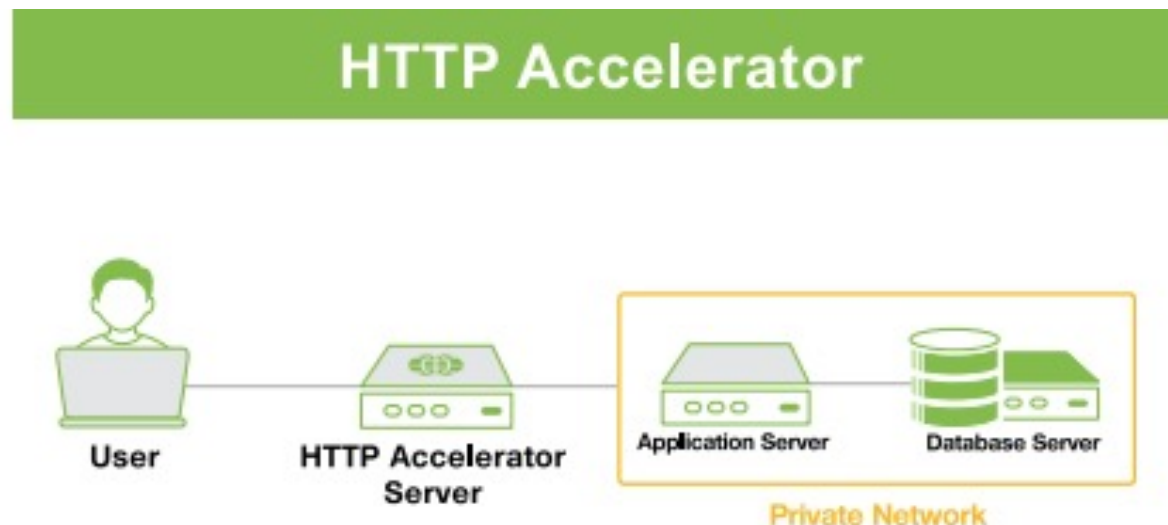
- ⦿ Useful in an environment that requires scaling by adding more servers (horizontal scaling).
- ⦿ Can protect against DDOS attacks by limiting client connections to a sensible amount and frequency

Cons

- ⦿ The LB can become a performance bottleneck if it does not have enough resources, or if it is configured poorly
- ⦿ Can introduce complexities that require additional consideration, e.g. where to perform SSL termination
- ⦿ The LB is a single point of failure - if it goes down, your whole service can go down

HTTP Accelerator

- A HTTP accelerator, or caching HTTP reverse proxy, can be used to reduce the time it takes to serve content to a user through a variety of techniques. Typically the response from a web or application server is cached for a period of time in memory.
- Examples of software capable of HTTP acceleration: Varnish, Squid, Nginx.



HTTP Accelerator

Pros

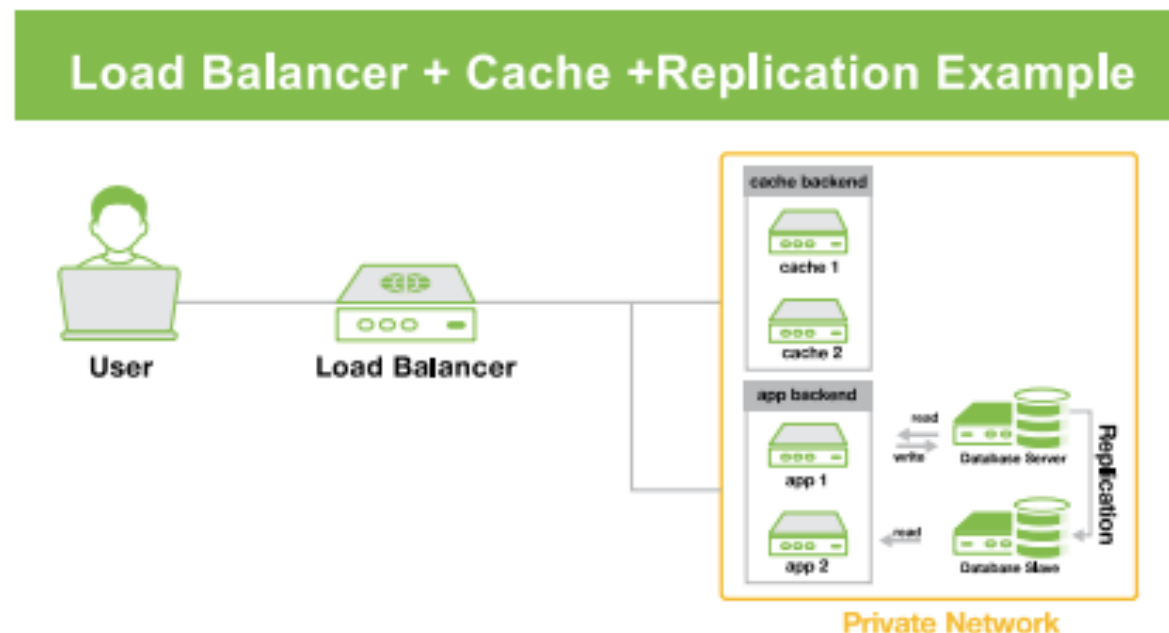
- ⦿ Increase site performance by reducing CPU load on web server, through caching and compression, thereby increasing user capacity
- ⦿ Can be used as a reverse proxy load balancer
- ⦿ Depending on implementation, can protect against DDOS attacks

Cons

- ⦿ Requires lots of tuning
- ⦿ If the cache-hit rate is low, it can reduce performance
- ⦿ Can be a single point of failure

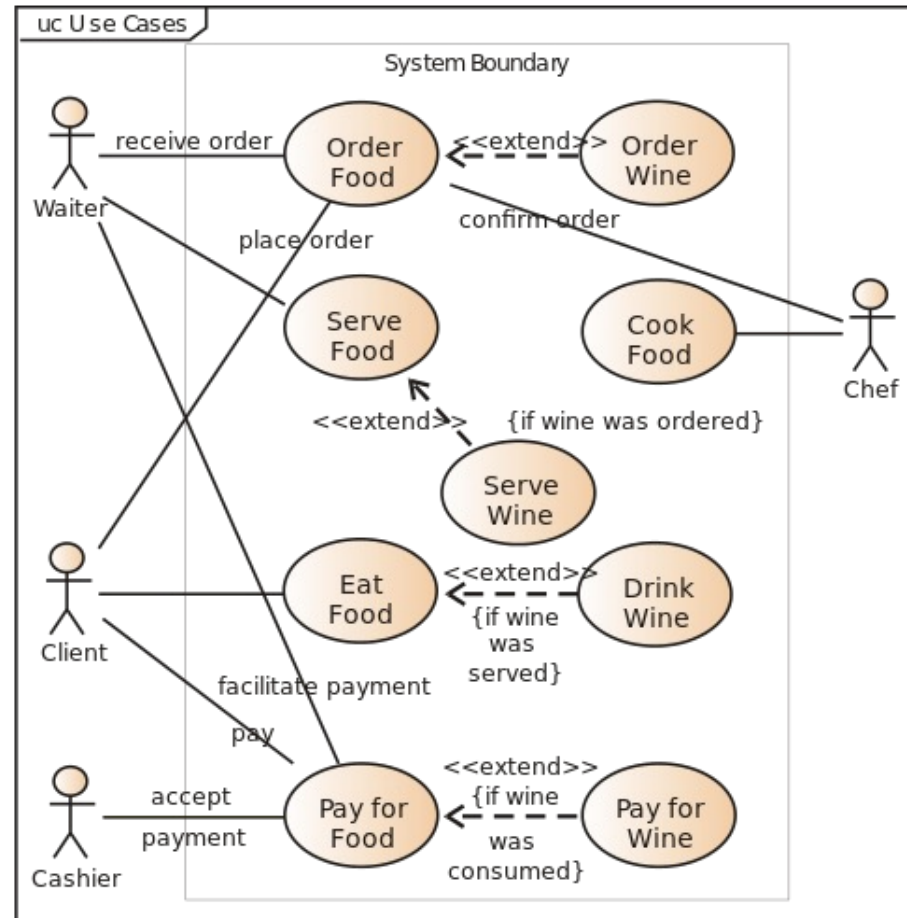
Combinations

- It is possible to load balance the caching servers, in addition to the application servers, and use database replication in a single environment. The purpose of combining these techniques is to reap the benefits of each without introducing too many issues or complexity.



Web Development Process

UML Use Case Diagram



Use case model of a restaurant business

Source: Wikimedia

UML Case Documentation – Flow of Events

- Create a big table that lists the use cases (see Monopoly example on Canvas).
- Describe each use case using fields such as:
 - **Name:** Give the use case a short name.
 - **Goal:** The main goal that the use case is trying to achieve.
 - **Actors:** Actors which initiate a use case and/or are involved in a use case.
 - **Preconditions:** A list of conditions that must be met before the use case can start.
 - **Main flow:** A full description of the use case.
 - **Subflows:** A list of actions or use cases on which the use case depends.
 - **Alternative Flows:** Any alternative action flows, e.g. ones that might come from error scenarios.
 - **Priority:** An indication of how important the use case is to the functioning of the system, or the MVP.

Use Case Documentation – Flow of Events

X Flow-of-Events for the <name> Use Case

X.1 Preconditions. *What needs to happen (in another use case) before this use case can start? What state must the system be in before the use case?*

X.2 Main Flow. *The main flow is a series of declarative steps.*

X.3 Sub-flows. *Sub-flows break down the main flow and other sub-flows to improve document readability.*

X.4 Alternative Flows. *The alternative flows define exceptional behavior that can interrupt the normal flow. Often alternative flows indicate what is to be done under error conditions. To determine alternative flows, ask yourself, “What could possibly go wrong?” for each of the actions in the main flow and the sub-flows.*

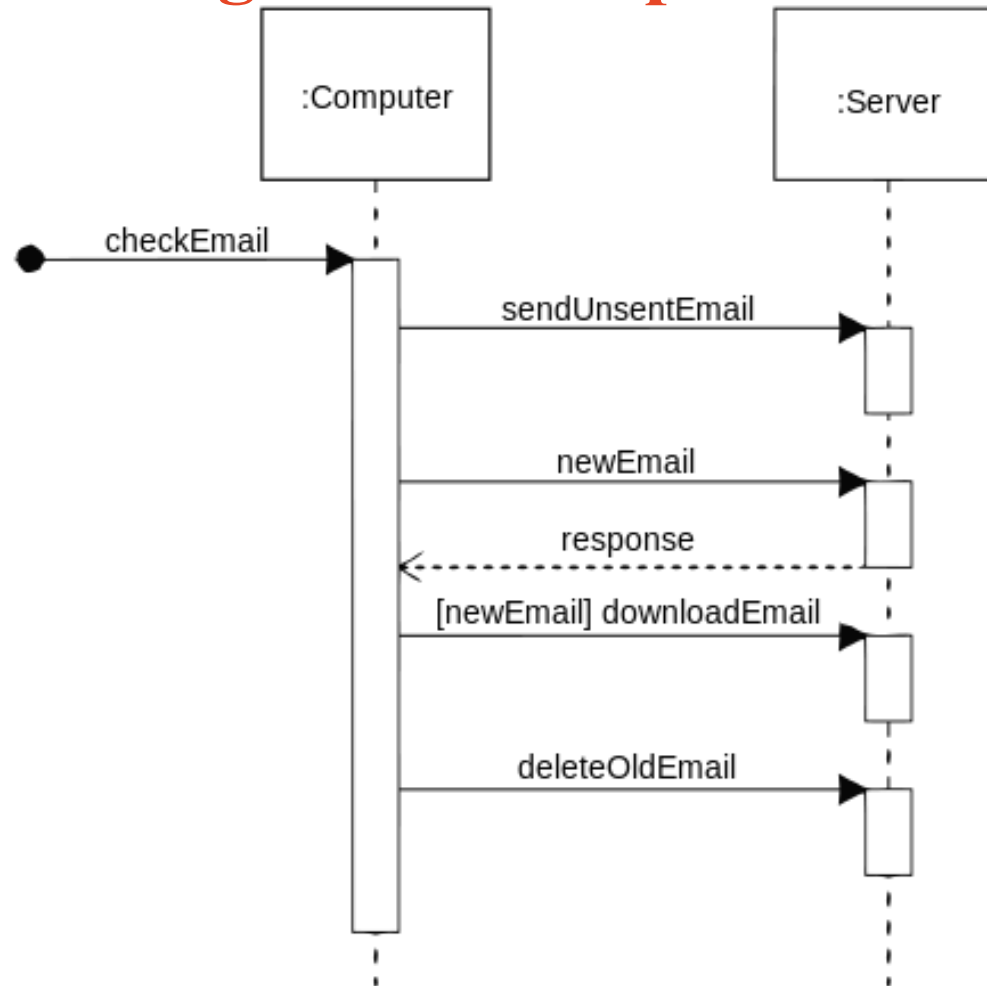
Note: X is a unique identifier for each use case.

Figure 4: Use Case Flow-of-Events Template

Sequence Diagrams Overview

- Sequence diagrams describe how objects interact with each other and in what order.
- Sequence diagrams are very useful for communicating how several systems interact, or providing detail around how various actors interact with system components in a multi-step process.

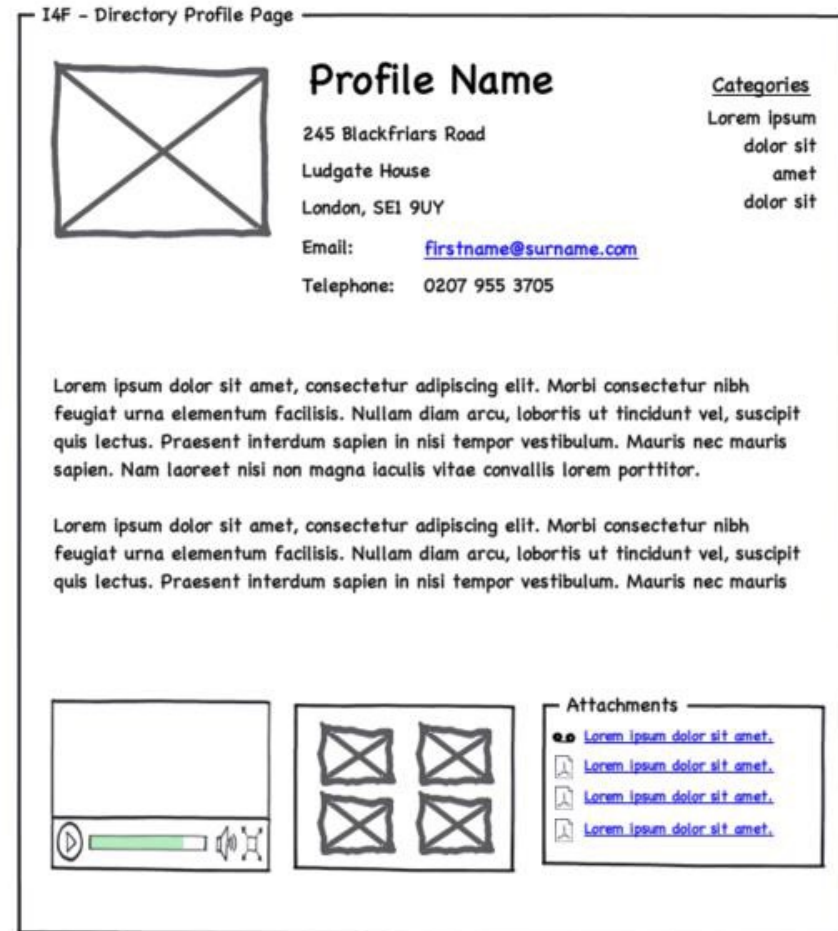
Sequence Diagrams Example



Example sequence diagram that shows a computer asking a server for emails
Source: Wikimedia

Wireframes

- Wireframing allows designers to gain a visual „feel“ for the site, without focussing on graphical detail.
- The idea is to create a „cartoon“ version of the website that enables a preliminary website experience.
- Wireframes are usually just a series of images, but can sometimes include clicking and linkage between the images in the form of a storyboard.



created with Balsamiq Mockups - www.balsamiq.com

Profile Page Wireframe

Source: [Wikimedia](https://commons.wikimedia.org/wiki/File:Profile_Page_Wireframe.png)

Software Requirements Specification

A Software Requirements Specification (SRS) document details exactly what the system does and does not do.

In essence, the SRS is a communication tool between project stakeholders and software designers.

It is a very important document that allows you to say the project is “finished”, and mitigates scope creep. This forms the primary specification document and should capture all information regarding the project.

The diagrams should not introduce new information that is not contained within the SRS.

Web Programming

HTML

HyperText Markup Language (HTML) is a markup language used for describing web pages.

```
<!DOCTYPE html>
<html>
  <head>
    <title>This is a title</title>
  </head>
  <body color="#333333">
    <p>Hello world!</p>
  </body>
</html>
```

HTML DTD

The [HTML5](#) does not have a DTD, hence the short DOCTYPE.

HTML 4's DOCTYPE is a bit more verbose:

```
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN"
    "http://www.w3.org/TR/html4/strict.dtd">
```

An HTML Custom element consists of...

- Changing HTML content
- Updating values of an attribute
- Dynamic HTML content (i.e., Date())
- Write directly to HTML output stream
- ...

A JS class

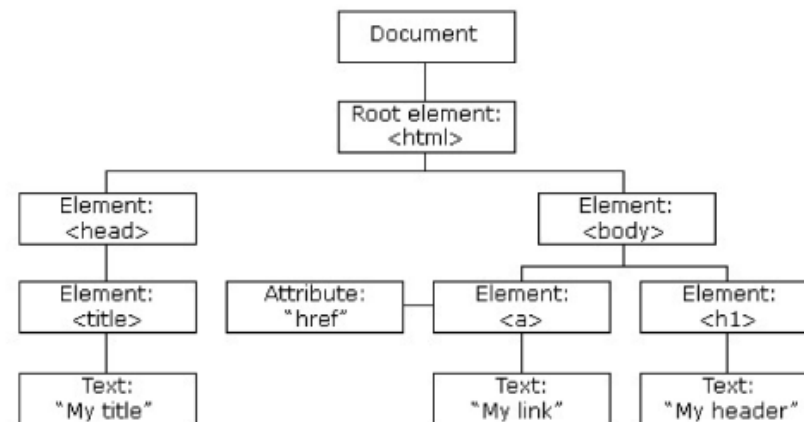
+

A tag name

```
class MyElement extends HTMLElement {  
    constructor() {  
        super();  
    }  
}  
  
// Register <my-element>  
customElements.define("my-element", MyElement);
```

DOM

DOM: a tree-like data structure forming HTML



The document object model treats a HTML document as a tree-like structure of objects.

Manipulating the DOM is possible from JavaScript, and forms the majority of JavaScript usage in the browser.

HTTP Error Codes

HTTP code category	Definition	Example
1xx	Informational	100: Continue 101: Switching Protocols
2xx	Success	200: OK – the standard response
3xx	Redirection	301: Moved Permanently 302: Found / Moved Temporarily (similar to 303 & 307) 304: Not Modified
4xx	Client	404: Not Found
5xx	Server	502: Bad gateway

Single Page Application (SPA)

Single page applications load a single HTML document and use JS to update the user's view without a hard refresh.

Single page applications move a lot of work to the client-side (browser) and typically create a smoother, more fluid user experience. They allow for significantly more powerful web applications.

Pros:

- Provide a faster interface.
- Encourage separation of concerns.
- Reusable
- Allow for “serverless” browser applications.
- Simpler binding of data to views.
- Easy to serve from a CDN, decreasing wait time for users.
- Removes load from back end servers.

Cons:

- Typically require more work.
- Typically require more experienced developers due to complexity.
- New frontier. Frameworks are rapidly evolving.
- Pushes workload into the browser, bad for low-spec/mobile devices.

AJAX

Asynchronous JavaScript and XML

- Allows the page to be updated with external data without a full refresh
- AJAX is a fundamental aspect of the modern web.
- Traditionally we would use XHR. Clunky.
- JavaScript libraries like JQuery improved the situation.
- Native browser APIs, such as Fetch, are also an option where supported.

Grunt

- Grunt – a JavaScript task runner.
- Similar to a Makefile but for HTML front ends.
- Also a Node.js application – installed via npm.
- Automates boring, repetitive tasks:
 - Minification of images, CSS, JS, HTML
 - Dependency linking
 - Linting (error/style checking)
 - Testing
- Gulp is another popular JS task runner.

Web Request Overview

Client

1. Browser: Parse URL
2. Browser: Check cache
3. OS: Identify network
4. OS: Resolve using DNS
5. OS: Establish TCP connection to web server
6. Browser: Submit HTTP request to web server

Server

1. Load Balancers (LB) establish TCP connection and receive HTTP request.
2. LB software parses HTTP header, then routes requests to an application server.
3. App server executes back-end website logic, probably connecting to a database.
4. App server generates a response, and returns it via the TCP connection.

Client

1. Browser: Parse response
2. Browser: Fetch additional resources:
 - Fetch assets (CSS, JS, Images)
 - Establish websocket connection/s
 - Load browser plugins
3. Render page

Web Stacks – LAMP and LEMP

LAMP / LEMP stacks are the most ubiquitous stacks used for developing and deploying web applications. Due to their modular nature, the components can easily be swapped out.

Layers	LAMP	LEMP
Operating System (OS)	Linux	Linux
Web Server	Apache	Nginx
Database	MySQL (PostgreSQL)	MySQL (PostgreSQL)
Programming Language	PHP (Other language)	PHP (Other language)
URL for extended reading	https://en.wikipedia.org/wiki/LAMP_(software_bundle)	https://lemp.io/

Common Linux distros:

- Ubuntu LTS (long-term stable version) 18.04 LTS
- Debian / Fedora
- CentOS / RedHat Enterprise Linux 7, CentOS 8 coming soon (Aug/Sep??)
- Amazon Linux / Amazon Linux 2

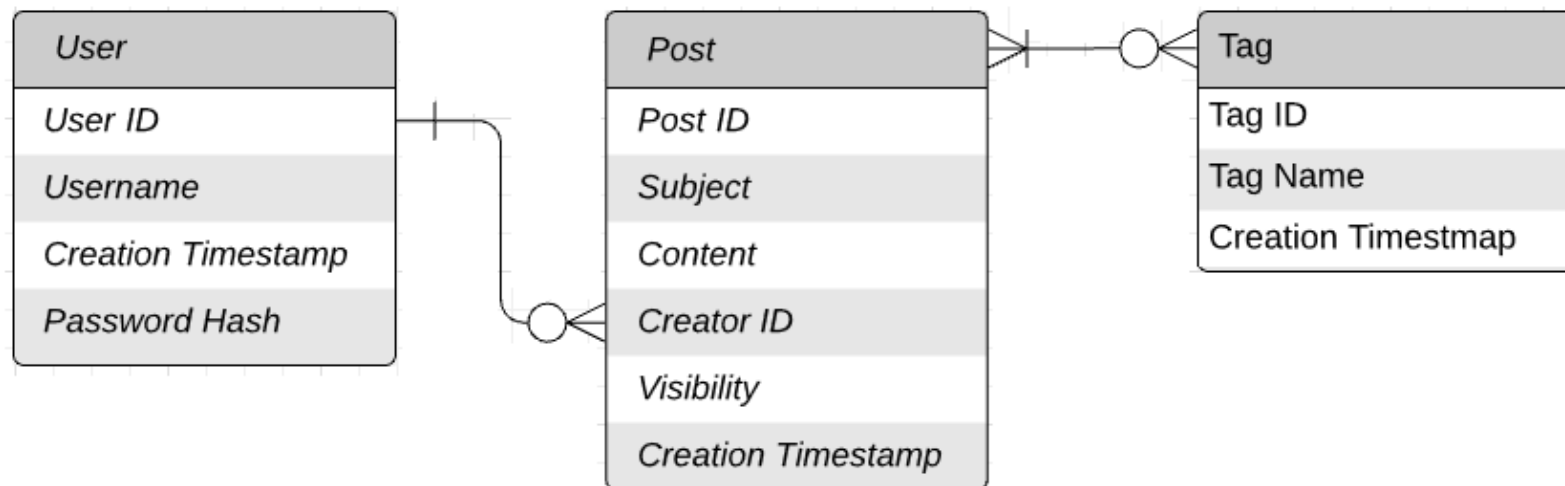
Databases

- The app servers typically read information from a databases.
- There are many types and flavours of database.
We will split them into break them into two main types:
- **SQL** Relational databases maintain and enforce relationships between data.
 - MySQL
 - PostgreSQL
 - SQLite
 - Microsoft SQL Server
- **NoSQL** Non-relational databases store data more like loose pages, and often without no schemas.
 - MongoDB
 - Redis
 - DynamoDB, Cloud Data Store
- Databases can reside on the same host as the app server, or separately (Recommended)

Data Modelling

- Data modelling is the process of defining and analysing data requirements of an information system.
- The process of data modelling involves:
 1. Identifying the detailed system requirements
 2. Normalising the entities
 3. Writing the data model
- Use UML to develop a formalised entity-relationship diagram (ERD), using Crow's foot notation.

Entity Relationship Diagram (ERD)



CI/CD Pipeline

What Does Continuous Integration Actually Do?

- Linting and static analysis
- Building
- Testing
 - Unit testing
 - Integration testing
 - Acceptance testing
 - Regression testing
- Publishing, Releasing and Deploying

Unit Testing

Unit, or functional, testing is the process of verifying functionality and components independently of the whole system.

The aim is to test the smallest piece of code that can be isolated, ideally a single function/method/subroutine.

One approach is to mock out other dependencies.

Example: testing the validate email function.

Integration Testing

Integration testing is the process of verifying the interactions/communication between software components.

A component is usually defined as a module or class. Stubs and drivers are commonly used when testing at this level.

System Testing

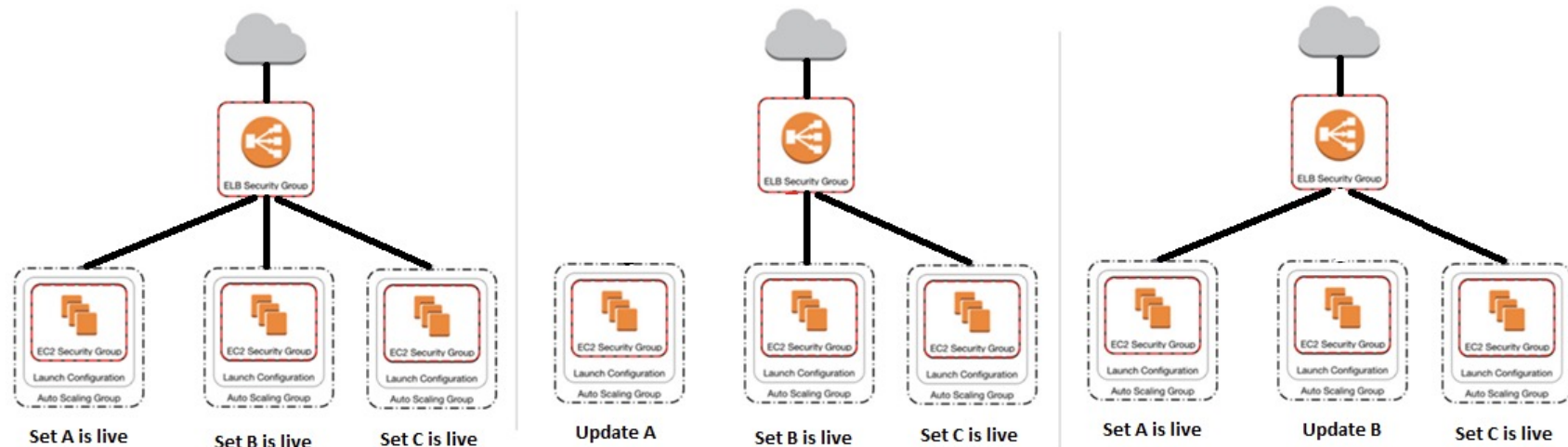
System testing is the process of verifying the behaviour and functionality of the entire system, including:

- security
- performance
- reliability
- external interfaces

Also commonly referred to as end-to-end testing.

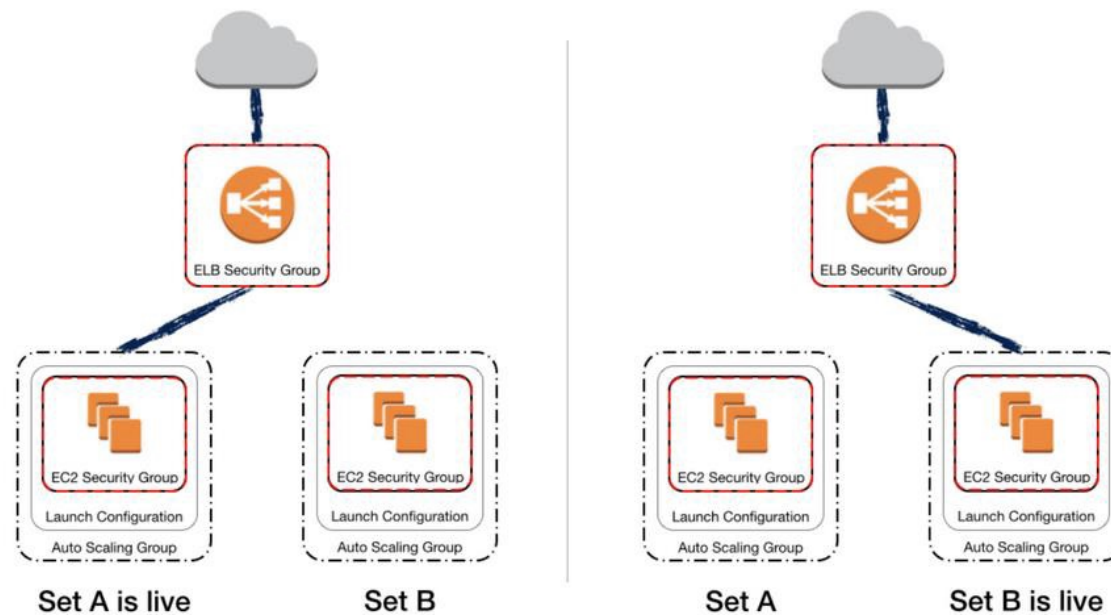
Types of Deployment

- Rolling deployment
 - Progressively deploy/release the new version until the entire pool has been updated.
 - Will serve a mix of old/new code/pages.



Types of Deployment

- Blue-green deployment
 - Do a hard switch over of all traffic to new infra/code



Version Control

- You should be familiar with Git, as well as how to use its most common features – push, commit, revert.

Web Security

Web Vulnerabilities Overview

- You should have an understanding about these vulnerabilities regarding (a) what they steal (b) how they attack (c) how to prevent
- SQLi
- XSS
- CSRF
- RCE
- L/RFI
- DDoS
- Authentication

Security Miscellaneous

- You should know how to store passwords and securely perform authentication.
- You should understand the risks of running outdated software.
- You should be familiar with DDOS attacks and mitigations.

**You reached the end of the lecture.
Also, the last lecture.**